

week6 예습과제

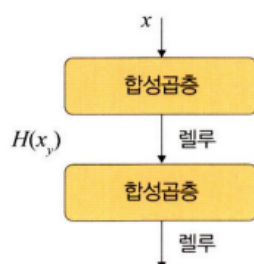
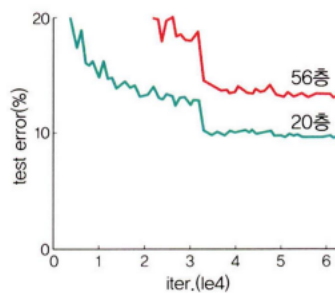
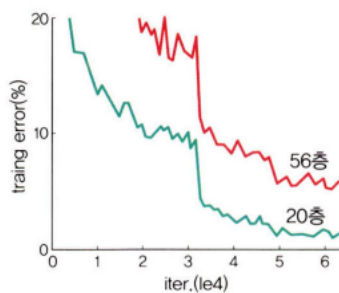
≡ 다중 선택	week6
≡ 다중 선택 1	

6.1 이미지 분류를 위한 신경망

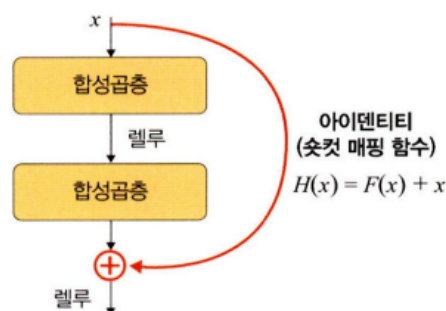
6.1.5 ResNet

ResNet의 등장 배경

- 마이크로소프트에서 개발, 논문 "*Deep Residual Learning for Image Recognition*"
- 핵심: 깊은 신경망을 효과적으로 학습시키기 위해 **레지듀얼(residual)** 개념 고안
- 문제점: 신경망이 깊어질수록 성능이 좋아지다가 일정 단계를 넘으면 오히려 성능 ↓
 - 56층 네트워크가 20층보다 train error, test error 모두 더 나쁨
 - 깊이 ↑ ≠ 성능 무조건 ↑
- GoogLeNet(2014, 22층) vs ResNet(152층) → 층이 깊어 기울기 소멸 문제 발생 가능
- 해결: 레지듀얼 블록(residual block) 도입 → 기울기가 잘 전파되도록 숏컷(shortcut, skip connection) 생성



기존 방식



ResNet

- 기존 방식: 입력 $x \rightarrow$ 합성곱층 $\rightarrow$ ReLU $\rightarrow$ 합성곱층 $\rightarrow H(x)$

- ResNet: $H(x) = F(x) + x$
 - $F(x)$: 합성곱층을 통과한 결과
 - x : 아이덴티티(숏컷 매핑 함수)로 그대로 전달된 입력

1) 블록(block)과 ResNet 구조

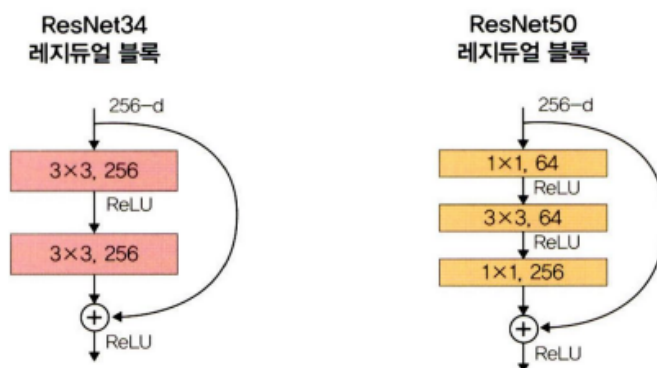
- **블록**: 계층의 묶음 (엄밀히는 합성곱층을 하나로 묶은 것)
- **레지듀얼 블록**: 묶인 계층들 + 숏컷
- **ResNet** = 레지듀얼 블록을 여러 개 쌓은 것
- **ResNet** $\approx$ VGG19 뼈대 + 합성곱층 추가(더 깊게) + 숏컷 추가



2) 기본 블록 vs 병목 블록

- **문제**: 계층을 계속 쌓으면 **파라미터 수 폭증**
 - ex) ResNet34: 합성곱층 34개, 블록 16개
- **해결**: 병목 블록(bottleneck block)

구분	기본 블록 (Basic Block)	병목 블록 (Bottleneck Block)
사용 모델	ResNet18, ResNet34	ResNet50, ResNet101, ResNet152
구성	3×3 합성곱 2개	1×1 → 3×3 → 1×1 합성곱
파라미터 수	많음	적음 (깊이가 깊어도 감소)



- **병목 블록이 파라미터를 줄이는 원리**
 - 3×3 합성곱층 **앞뒤로 1×1 합성곱층 배치**
 - 1×1 합성곱으로 **채널 수를 줄였다(차원 축소) → 3×3 연산 → 다시 늘림(차원 확장)**
 - 이 부분이 병목(좁은 목)과 같다 해서 **병목 블록**

- 병목 블록의 장점
 - 계층을 더 깊게 쌓으면서 **계산 비용 ↓**
 - 계층 ↑ = 활성화 함수 ↑ → **비선형성(non-linearity) 처리 능력 ↑** → 다양한 입력 데이터 처리 가능



파라미터 수 참고 (256채널 입력 기준)

- 기본 블록: $3 \times 3 \times 256 \times 256 \times 2 \approx 1.18M$
- 병목 블록: $(1 \times 1 \times 256 \times 64) + (3 \times 3 \times 64 \times 64) + (1 \times 1 \times 64 \times 256) \approx 0.07M$
- 교재 본문에는 39.3216M / 6.9632M으로 표기되어 있으나 직접 계산하면 위 값 (소수점 위치가 다르게 인쇄된 것으로 보임). 핵심은 **병목 블록의 파라미터가 훨씬 적다**는 점

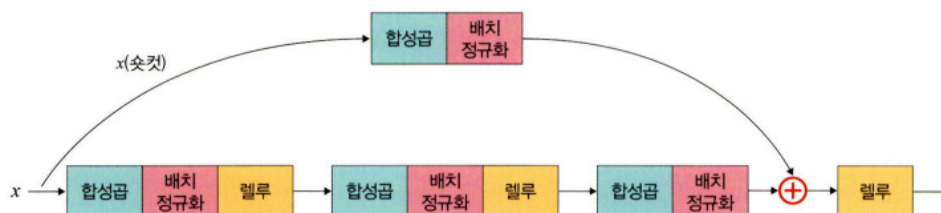
3) 아이덴티티 매핑 (identity mapping)

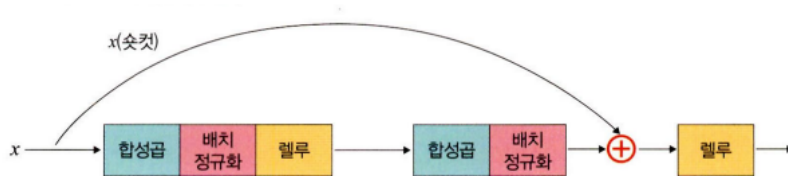
- 다른 이름: **숏컷(shortcut)**, **스킵 연결(skip connection)**
- 입력 x 가 어떤 함수를 통과하더라도 **다시 x 라는 형태로 더해지도록** 하는 것
- 블록 마지막에 처음 입력 x 를 더해줌 ($x += i$) → 기울기 소멸 문제 방지
- 예) x 가 (28, 28, 64)이면 합성곱층을 통과해도 (28, 28, 64) → 같은 형태끼리 더함

4) 다운샘플 (downsample)

- 역할: **특성 맵 크기를 줄임** (풀링과 유사)
- **필요한 이유**: 블록 간 특성 맵 형태가 다르면 아이덴티티 매핑(덧셈)을 할 수 없음
 - 예) 보라색 영역 (28, 28, 64) → 노란색 영역 시작 지점: 채널 128로 증가, $/2$ 스트라이드 2 → (14, 14, 128)
- **방법**: 숏컷 쪽에 **스트라이드 2를 가진 1×1 합성곱층** 연결 → 입력을 출력 형태에 맞춤

구분	아이덴티티 블록	합성곱 블록 (프로젝션 숏컷)
입출력 차원	입력 = 출력	입력 $\neq$ 출력
숏컷	입력 x 를 그대로 더함	입력 x 에 1×1 합성곱 + BN 적용 후 더함





5) 구현 코드 (핵심)

① 기본 블록 (BasicBlock)

```
class BasicBlock(nn.Module):
    expansion = 1
    def __init__(self, in_channels, out_channels, stride=1, downsample=False):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=
3,
                                stride=stride, padding=1, bias=False)
        # 3x3 합성곱
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size
=3,
                                stride=1, padding=1, bias=False)
        # 3x3 합성곱
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.relu = nn.ReLU(inplace=True)

        if downsample: # 입력 크기가 다르면 1x1 합성곱으로 숏컷 맞춤
            conv = nn.Conv2d(in_channels, out_channels, kernel_size=1,
                                stride=stride, bias=False)
            bn = nn.BatchNorm2d(out_channels)
            downsample = nn.Sequential(conv, bn)
        else:
            downsample = None
        self.downsample = downsample

    def forward(self, x):
        i = x
        x = self.relu(self.bn1(self.conv1(x)))
        x = self.bn2(self.conv2(x))
        if self.downsample is not None:
            i = self.downsample(i) # 다운샘플 적용
        x += i # 아이덴티티 매핑
```

```
x = self.relu(x)
return x
```

② 병목 블록 (Bottleneck)

```
class Bottleneck(nn.Module):
    expansion = 4 # 출력 채널을 out_channels의 4배로 확장
    def __init__(self, in_channels, out_channels, stride=1, downsample=False):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=1,
                                stride=1, bias=False)
        # 1x1 (차원 축소)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3,
                                stride=stride, padding=1, bias=False)
        # 3x3
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.conv3 = nn.Conv2d(out_channels, self.expansion*out_channels,
                                kernel_size=1, stride=1, bias=False)
        # 1x1 (차원 확장)
        self.bn3 = nn.BatchNorm2d(self.expansion*out_channels)
        self.relu = nn.ReLU(inplace=True)

        if downsample:
            conv = nn.Conv2d(in_channels, self.expansion*out_channels,
                              kernel_size=1, stride=stride, bias=False)
            bn = nn.BatchNorm2d(self.expansion*out_channels)
            downsample = nn.Sequential(conv, bn)
        else:
            downsample = None
        self.downsample = downsample

    def forward(self, x):
        i = x
        x = self.relu(self.bn1(self.conv1(x)))
        x = self.relu(self.bn2(self.conv2(x)))
        x = self.bn3(self.conv3(x))
        if self.downsample is not None:
            i = self.downsample(i)
        x += i
```

```
x = self.relu(x)
return x
```

③ ResNet 전체 네트워크

```
class ResNet(nn.Module):
    def __init__(self, config, output_dim, zero_init_residual=False):
        super().__init__()
        block, n_blocks, channels = config          # config 값 분리
        self.in_channels = channels[0]
        assert len(n_blocks) == len(channels) == 4

        self.conv1 = nn.Conv2d(3, self.in_channels, kernel_size=7,
                                stride=2, padding=3, bias=False)
        self.bn1 = nn.BatchNorm2d(self.in_channels)
        self.relu = nn.ReLU(inplace=True)
        self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=
1)

        self.layer1 = self.get_resnet_layer(block, n_blocks[0], channe
ls[0])
        self.layer2 = self.get_resnet_layer(block, n_blocks[1], channe
ls[1], stride=2)
        self.layer3 = self.get_resnet_layer(block, n_blocks[2], channe
ls[2], stride=2)
        self.layer4 = self.get_resnet_layer(block, n_blocks[3], channe
ls[3], stride=2)

        self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
        self.fc = nn.Linear(self.in_channels, output_dim)

        if zero_init_residual:  # 레지듀얼 분기의 마지막 BN 가중치를 0으로
초기화
            for m in self.modules():
                if isinstance(m, Bottleneck):
                    nn.init.constant_(m.bn3.weight, 0)
                elif isinstance(m, BasicBlock):
                    nn.init.constant_(m.bn2.weight, 0)

    def get_resnet_layer(self, block, n_blocks, channels, stride=1):
        layers = []
        # in_channels와 출력 채널이 다르면 downsample 적용
        downsample = self.in_channels != block.expansion * channels
        layers.append(block(self.in_channels, channels, stride, downsa
```

```

mple))
    for i in range(1, n_blocks):
        # n_blocks만큼 블록 추가
        layers.append(block(block.expansion*channels, channels))
    self.in_channels = block.expansion * channels
    return nn.Sequential(*layers)

def forward(self, x):
    x = self.conv1(x)    # 224 x 224
    x = self.bn1(x)
    x = self.relu(x)
    x = self.maxpool(x)  # 112 x 112
    x = self.layer1(x)   # 56 x 56
    x = self.layer2(x)   # 28 x 28
    x = self.layer3(x)   # 14 x 14
    x = self.layer4(x)   # 7 x 7
    x = self.avgpool(x)  # 1 x 1
    h = x.view(x.shape[0], -1)
    x = self.fc(h)
    return x, h

```



zero_init_residual

- 각 레지듀얼 분기(residual branch)의 **마지막 BN을 0으로 초기화** → 다음 레지듀얼 분기를 0에서 시작
- 모델 생성·학습과는 무관하지만, 논문(arXiv:1706.02677)에 따르면 **성능이 0.2~0.3% 향상** → ResNet에서 많이 사용
- 레지듀얼 분기: 조건에 따라 A, B, C 등으로 갈라지는 분기와 같은 개념

④ ResNet 종류별 Config

```

from collections import namedtuple
ResNetConfig = namedtuple('ResNetConfig', ['block', 'n_blocks', 'channels'])

# 기본 블록
resnet18_config = ResNetConfig(block=BasicBlock, n_blocks=[2, 2, 2, 2],
                                channels=[64, 128, 256, 512])
resnet34_config = ResNetConfig(block=BasicBlock, n_blocks=[3, 4, 6, 3],
                                channels=[64, 128, 256, 512])
# 병목 블록
resnet50_config = ResNetConfig(block=Bottleneck, n_blocks=[3, 4, 6, 3],

```

```
channels=[64,128,256,512])
resnet101_config = ResNetConfig(block=Bottleneck, n_blocks=[3,4,23,3],
channels=[64,128,256,512])
resnet152_config = ResNetConfig(block=Bottleneck, n_blocks=[3,8,36,3],
channels=[64,128,256,512])
```

- `namedtuple`: 튜플 성질 + 인덱스뿐 아니라 키 값(이름)으로도 접근 가능

⑤ 모델 생성 / 사전 훈련 모델

```
OUTPUT_DIM = 2 # 개, 고양이 2개 클래스
model = ResNet(resnet50_config, OUTPUT_DIM)

# 사전 훈련된 ResNet (코드 한 줄)
pretrained_model = models.resnet50(pretrained=True)
```

- 직접 만든 네트워크와 사전 훈련된 네트워크 구조는 동일 (fc의 out_features만 1000 vs 2)
- 구조를 알고 쓰는 것이 중요 → 네트워크 수정, 입출력 텐서 크기 파악에 도움

⑥ 학습 결과 요약

- 옵티마이저 Adam(lr=1e-7), 손실함수 CrossEntropyLoss, 10 epoch
- 정확도 ≈ 50% 수준으로 낮음 (데이터가 적기 때문, 훈련 400 / 검증 92 / 테스트 10)
- 이 예제의 목적은 성능 향상 × → **CNN 모델의 네트워크 구성/사용법 학습** ○
- Acc @1: 모델이 첫 번째로 예측한 레이블의 정확도 / Acc @5(코드에서는 k=2): 상위 k개 안에 정답이 있는 비율 (클래스가 2개라 100%)

6.2 객체 인식을 위한 신경망

객체 인식 (Object Detection)

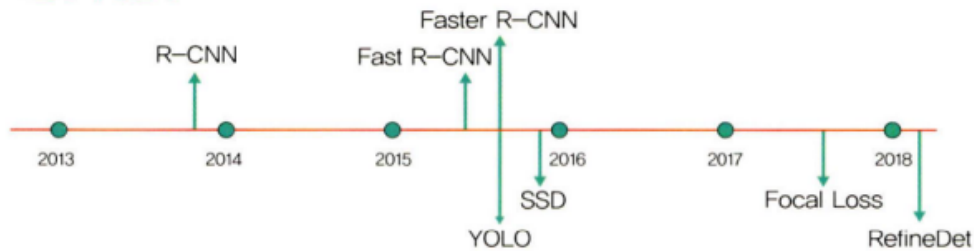
: 이미지/영상 내 객체를 식별하는 컴퓨터 비전 기술

- 객체 인식 = 분류(classification) + 위치 검출(localization)
 - 분류: 각 객체가 무엇인지
 - 위치 검출: 객체 위치를 바운딩 박스(bounding box) 로 표시
- 활용: 자율 주행 자동차, CCTV, 무인 점포 등

구분	1단계 객체 인식 (1-stage)	2단계 객체 인식 (2-stage)
방식	분류와 위치 검출을 동시에	분류와 위치 검출을 순차적으로
속도	빠름	느림
정확도	낮음	높음

구분	1단계 객체 인식 (1-stage)	2단계 객체 인식 (2-stage)
대표 모델	YOLO 계열, SSD 계열	R-CNN 계열 (R-CNN → Fast R-CNN → Faster R-CNN)

2단계 객체 인식

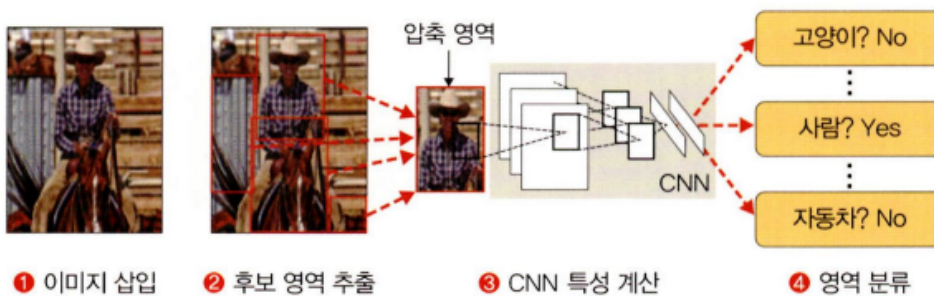


1단계 객체 인식

6.2.1 R-CNN

- 과거: 슬라이딩 윈도우 방식 (일정 크기 윈도우로 이미지 전체 탐색) → 비효율적이라 잘 안 씀
- 현재: 선택적 탐색(selective search) 으로 후보 영역(region proposal) 추출
- R-CNN (Region-based CNN) = 이미지 분류 CNN + 후보 영역 알고리즘

R-CNN 학습 절차



1. 이미지 입력
2. 선택적 탐색으로 2000개 바운딩 박스 추출 → 잘라내고(cropping) CNN 입력용으로 같은 크기 (227×227)로 통일(warping)
3. 크기가 동일한 2000개 이미지에 각각 CNN 적용
4. 각각 분류 진행 → 결과 도출



선택적 탐색 (Selective Search)

: 객체가 있을 만한 후보 영역을 알아내는 방법. 분할 방식으로 시드(seed) 선정 → 완전 탐색 적용

1. 초기 영역 생성 (sub-segmentation): 각 객체가 영역 1개에 할당되도록 이미지를 다수의 영역으로 분할

2. 작은 영역의 통합: 탐욕(greedy) 알고리즘으로 비슷한 영역을 하나로 통합 (반복)

3. 후보 영역 생성: 통합된 영역을 기반으로 바운딩 박스 추출

- 완전 탐색(exhaustive search): 크기·비율이 모두 다른 후보를 고려해 찾는 기법
- 분할(segmentation): 색상, 모양, 무늬 등 특성에 따라 분할
- 시드(seed): 분할 시 확장의 기준점이 되는 픽셀

• R-CNN의 단점

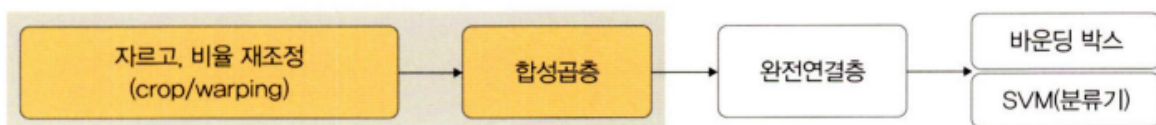
1. 3단계의 복잡한 학습 과정
2. 긴 학습 시간, 대용량 저장 공간
3. 객체 검출 속도 문제

⇒ 해결하기 위해 **Fast R-CNN** 등장

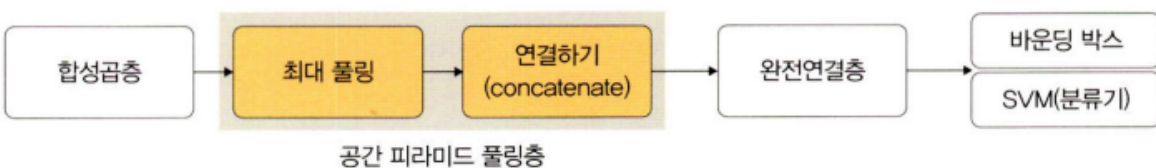
6.2.2 공간 피라미드 풀링 (Spatial Pyramid Pooling, SPP)

- 기존 문제: 완전연결층 때문에 입력 이미지 크기를 고정해야 함 → crop/warp 시 물체 일부가 잘리거나 모양이 변형
- SPP: 입력 이미지 크기에 관계없이 합성곱층 통과 → 완전연결층 전에 특성 맵을 동일한 크기로 조절하는 풀링층 적용
- 장점
 - 원본 이미지 크기를 조절하지 않아 특징 훼손 ×
 - 이미지 분류, 객체 인식 등 여러 작업에 적용 가능

R-CNN



공간 피라미드 풀링



6.2.3 Fast R-CNN

- R-CNN 문제: 바운딩 박스마다 CNN을 돌림 + 분류에 긴 학습 시간
- 개선: RoI 풀링 도입
 - 선택적 탐색에서 찾은 바운딩 박스 정보가 CNN을 통과하면서 유지

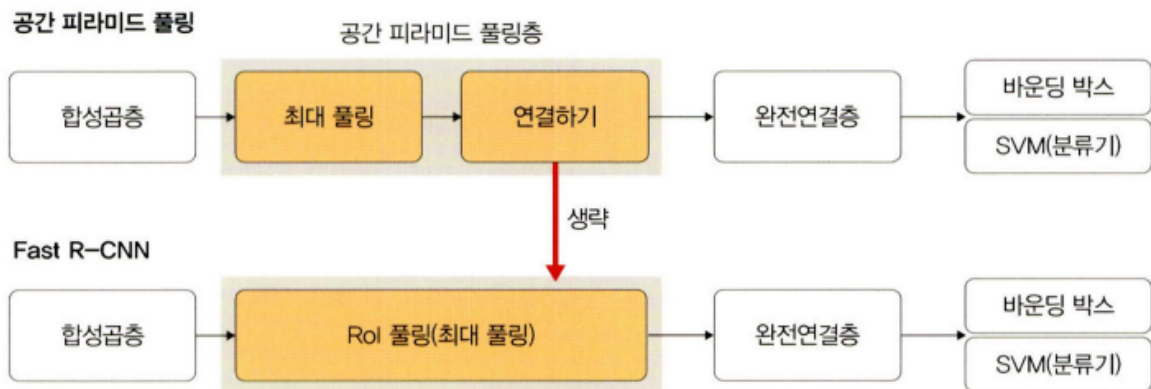
- 최종 CNN 특성 맵에 풀링을 적용해 완전연결층에 맞게 크기 조정
- ⇒ 바운딩 박스마다 CNN을 돌리는 시간 단축



RoI 풀링 (RoI pooling)

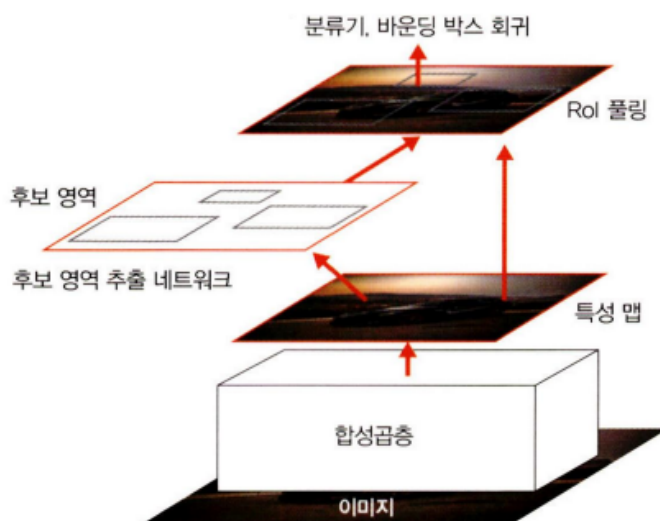
: 크기가 다른 특성 맵 영역마다 스트라이드를 다르게 하여 최대 풀링 → 출력 크기를 동일하게 맞추는 방법

- 예) 8×8 특성 맵에서 선택적 탐색으로 뽑은 7×5 후보 영역을 2×2 로 만들기
 - 스트라이드: ($7/2 \approx 3$, $5/2 \approx 2$)로 풀링 영역 결정 → 최대 풀링 → 2×2 결과



6.2.4 Faster R-CNN

- Fast R-CNN의 속도 걸림돌 = 후보 영역 생성(외부 선택적 탐색, CPU 계산)
- 핵심: 후보 영역 추출 네트워크(Region Proposal Network, RPN) 추가 → 내부 네트워크(GPU 계산)에서 후보 영역 생성
- 구조: 이미지 → 합성곱층 → 특성 맵 → RPN → RoI 풀링 → 분류기 + 바운딩 박스 회귀
 - 바운딩 박스 회귀(bounding-box regression): 박스가 객체를 정확히 포함하도록 위치 보정

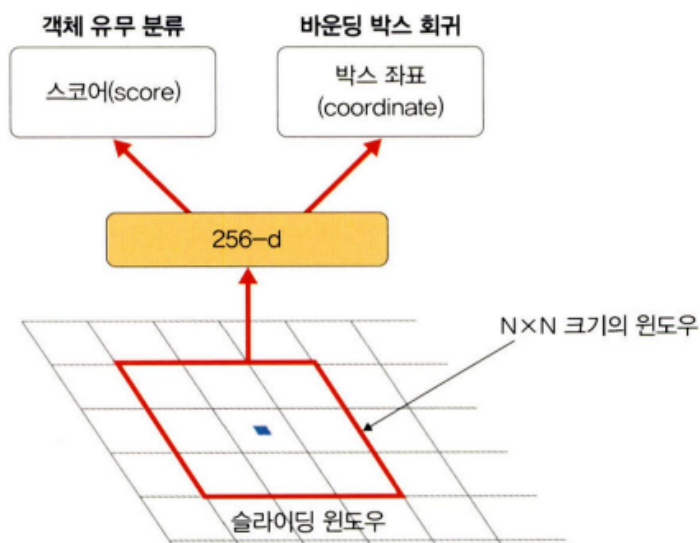


후보 영역 추출 네트워크 (RPN)

- 특성 맵에서 $N \times N$ 크기의 작은 윈도우를 입력으로 받음
 - 이진 분류: 객체 존재 유무 판단 (객체 유무 분류 스코어)
 - 바운딩 박스 회귀: 위치 보정 (박스 좌표)
- 슬라이딩 윈도우 방식으로 특성 맵 전체 탐색
- 한계: 고정된 $N \times N$ 입력만으로는 다양한 크기·비율의 객체 수용이 어려움

앵커 (anchor)

- 여러 크기·비율의 레퍼런스 박스(reference box) k 개를 미리 정의 → 슬라이딩 윈도우 위치마다 k 개 박스 출력
- RPN 출력
 - 분류: 객체/배경 판단 → $2k$ 개
 - 회귀: x, y, w, h 보정 → $4k$ 개
- 특성 맵 크기가 $w \times h$ 이면 앵커는 총 $w \times h \times k$ 개



6.3 이미지 분할을 위한 신경망

이미지 분할 (Image Segmentation)

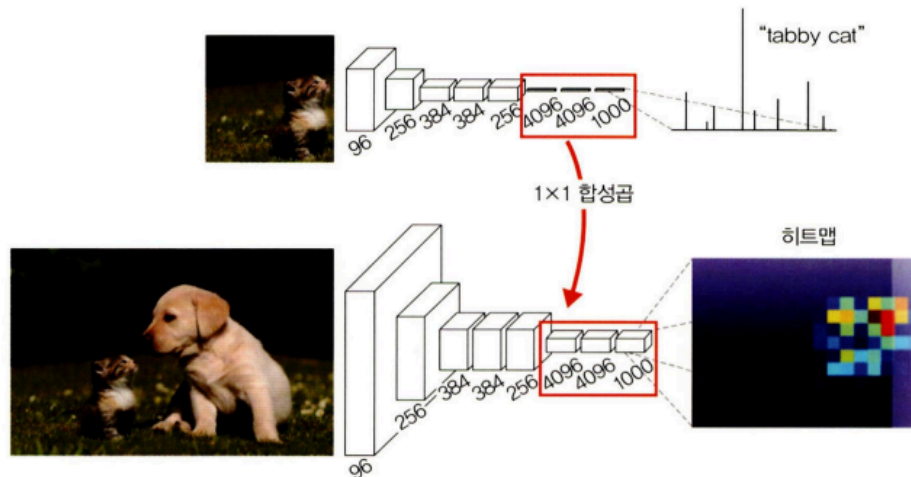
: 신경망을 훈련시켜 이미지를 픽셀 단위로 분할 → 이미지에 포함된 객체 추출

- 대표 네트워크: FCN, 합성곱 & 역합성곱 네트워크, U-Net, PSPNet, DeepLabv3/DeepLabv3+

6.3.1 완전 합성곱 네트워크 (Fully Convolutional Network, FCN)

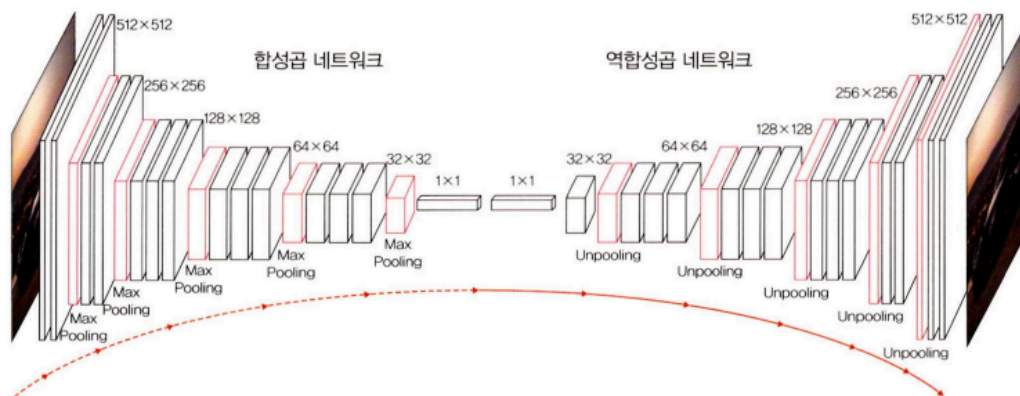
- 완전연결층의 한계

- 고정된 크기의 입력만 받을 수 있음
- 완전연결층을 거치면 위치 정보가 사라짐
- FCN: 완전연결층을 1x1 합성곱으로 대체
 - 이미지 분류용 CNN(AlexNet, VGG16, GoogLeNet)을 이미지 분할에 적합하게 변형
 - 예) AlexNet의 완전연결층 3개 → 1x1 합성곱 3개 → 위치 정보 유지 → 히트맵(heatmap) 으로 고양이 위치 확인
- 장점: 합성곱층만 쓰므로 입력 이미지 크기 제약 ×



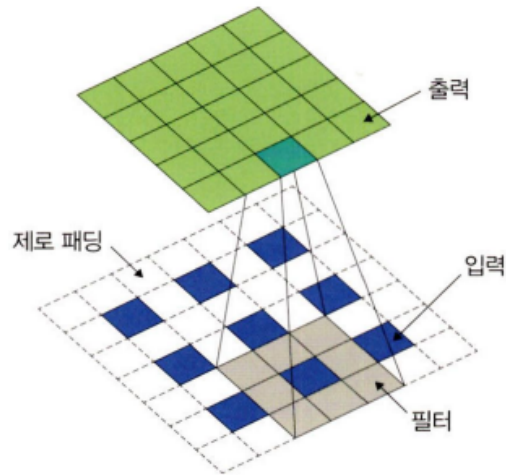
6.3.2 합성곱 & 역합성곱 네트워크

- FCN의 단점
 - 여러 합성곱층·풀링층을 거치며 해상도 ↓
 - 낮아진 해상도를 업 샘플링으로 복원하는 과정에서 세부 정보 손실
- ⇒ 역합성곱(deconvolution) 네트워크 도입
- 역합성곱: CNN 최종 출력을 원래 입력 이미지와 같은 크기로 복원할 때 사용 (= 업 샘플링), 시멘틱 분할 (semantic segmentation) 등에 활용



- 합성곱은 특성 맵 크기 ↓ ↔ 역합성곱은 특성 맵 크기 ↑

- 역합성곱 동작 방식



1. 각 픽셀 주위에 **제로 패딩** 추가
2. 패딩된 것에 **합성곱 연산** 수행

6.3.3 U-Net

: 바이오 메디컬 이미지 분할을 위한 합성곱 신경망

- 특징

1. 속도가 빠름
 - 기존 슬라이딩 윈도우: 이전 패치(patch, 이미지 인식 단위)에서 검증한 부분을 다음 패치에서 또 검증 → 느림
 - U-Net: 이미 검증이 끝난 패치는 건너뛴
2. 트레이드오프에 빠지지 않음
 - 패치 크기 ↑ → 컨텍스트(context) 인식 ◦, 지역화 × / 패치 크기 ↓ → 반대
 - U-Net은 컨텍스트 인식과 지역화의 트레이드오프를 개선



지역화 (localization)

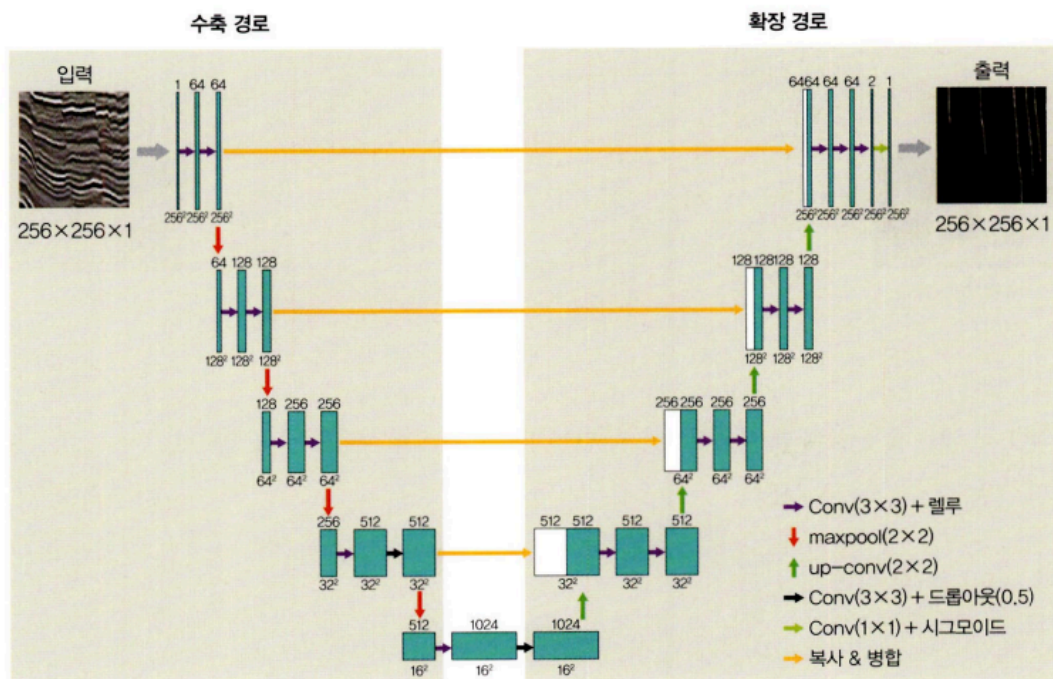
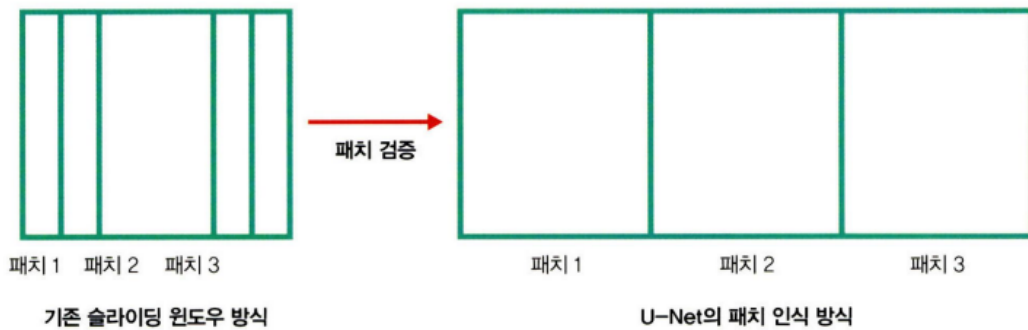
: 이미지 안 객체(예: 고양이)의 **위치 정보를 출력**하는 것. 주로 바운딩 박스를 사용하며, 왼쪽 위 (left top)·오른쪽 아래(right bottom) 좌표를 출력

- 구조: FCN 기반, 수축 경로(contracting path) + 확장 경로(expansive path)

구분	수축 경로 (왼쪽)	확장 경로 (오른쪽)
역할	컨텍스트 포착	특성 맵 업 샘플링 + 수축 경로의 특성 맵과 결합 → 정확한 지역화

구분	수축 경로 (왼쪽)	확장 경로 (오른쪽)
구성	3×3 합성곱 2개(사이에 드롭아웃) 블록 4개 + 블록마다 max pooling 으로 크기 ↓	블록 앞에 up-conv(2×2) 붙여 크기 ↑ + 수축 경로 특성 맵 복사 & 병합

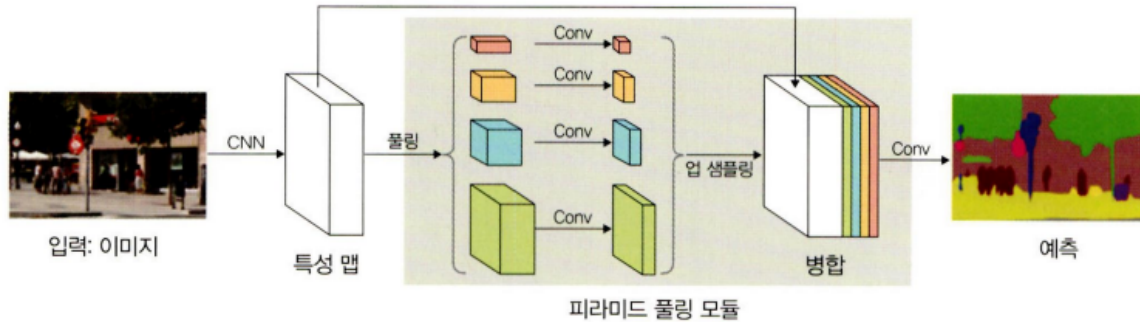
- 마지막 층: **1×1 합성곱 + 시그모이드**
- 크기가 다양한 객체를 분할하기 위해 **다운 샘플링과 업 샘플링을 순서대로 반복**하며 다양한 크기의 특성 맵을 병합



6.3.4 PSPNet (Pyramid Scene Parsing Network)

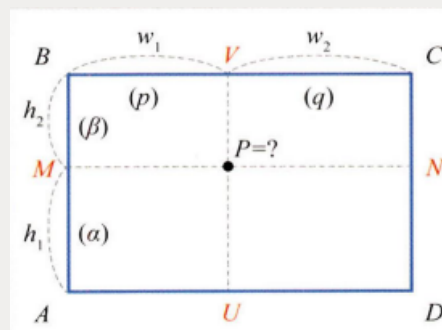
- CVPR 2017에서 발표된 **시멘틱 분할** 알고리즘
- 완전연결층의 한계 극복을 위해 **피라미드 풀링 모듈** 추가
- 과정
 1. 서로 다른 크기로 여러 차례 풀링 (1×1, 2×2, 3×3, 6×6)
 - 1×1 특성 맵 = 가장 광범위한 정보, 각 크기마다 서로 다른 영역 정보 담음
 2. **1×1 합성곱**으로 채널 수 조정 → 풀링층 개수가 N이면 **출력 채널 수 = 입력 채널 수 / N**

3. 모듈의 입력 크기에 맞게 **업 샘플링 (양선형 보간법 사용)**
 4. 원래 특성 맵과 1~3에서 만든 특성 맵들을 **병합(concat)** → 예측
- 풀링 개수(4개)는 구현에 따라 변경 가능



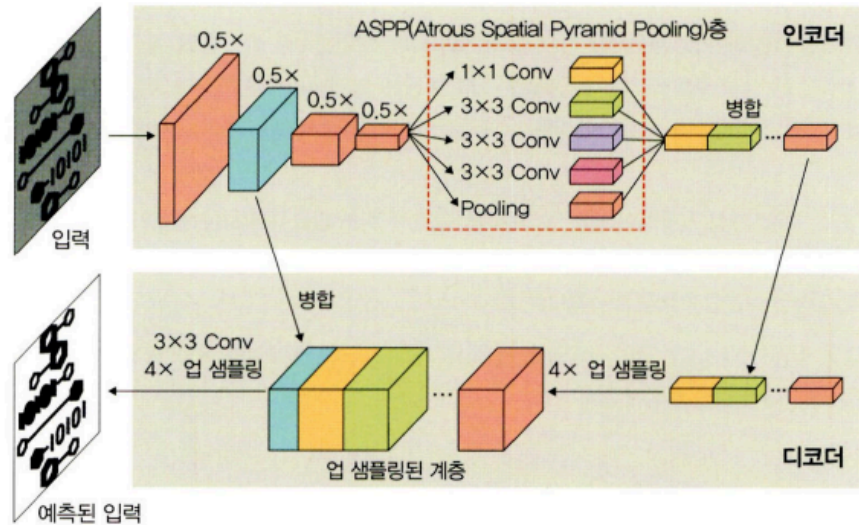
양선형 보간법 (Bilinear Interpolation)

- **보간법:** 화소 값을 할당받지 못한 빈 화소에 값을 할당해 좋은 품질의 영상을 만드는 방법
 - **선형 보간법:** 원시 영상 화소 값 **2개**로 새 화소 값 계산
 - **양선형 보간법:** 화소당 선형 보간 **3번** 수행, 새 화소는 **가장 가까운 화소 4개에 가중치를 곱해 합산**
- 직사각형 네 꼭지점(A, B, C, D) 값이 주어졌을 때 내부 점 P의 값
 - $P = q(\beta A + \alpha B) + p(\beta D + \alpha C)$
 - $\alpha = h_1 / (h_1 + h_2), \beta = h_2 / (h_1 + h_2), p = w_1 / (w_1 + w_2), q = w_2 / (w_1 + w_2)$
 - (w_1, w_2 : x축 방향 거리 / h_1, h_2 : y축 방향 거리)



6.3.5 DeepLabv3 / DeepLabv3+

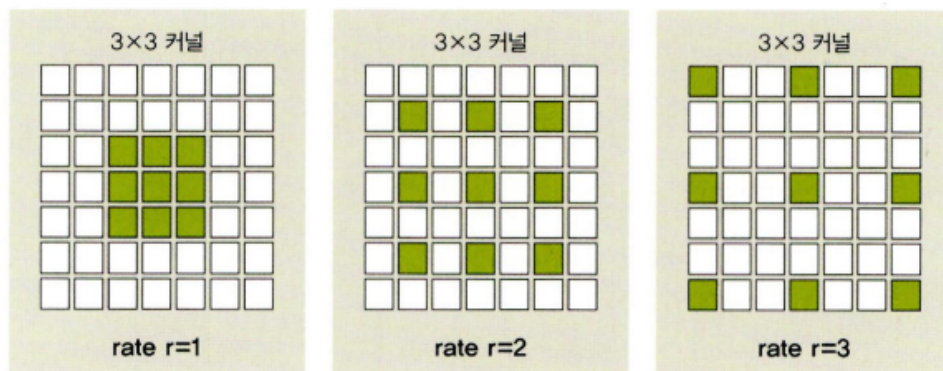
- 완전연결층의 단점 보완을 위해 **Atrous 합성곱** 사용
- 인코더-디코더 구조



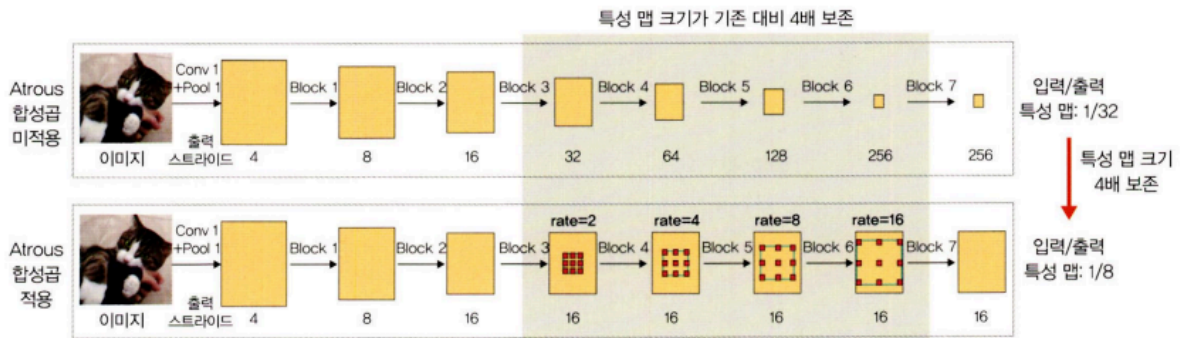
- 일반적인 인코더-디코더에서는 불가능했던 인코더 특성 맵의 해상도 제어를 Atrous 합성곱으로 가능하게 함

• ASPP (Atrous Spatial Pyramid Pooling)층 사용

Atrous 합성곱



- 필터 내부에 빈 공간을 둔 채로 작동
- **rate(r)**: 빈 공간의 양을 결정하는 파라미터
 - $r = 1 \rightarrow$ 기존 합성곱과 동일 (빈 공간 $\times$)
 - $r \uparrow \rightarrow$ 빈 공간 $\uparrow$ (3×3 커널 기준 $r=1, 2, 3$)
- 장점: 파라미터 수를 늘리지 않으면서 수용 영역(receptive field)을 크게 확장 $\rightarrow$ 이미지 분할에서 많이 사용
- 효과
 - 일반 CNN: 출력이 입력의 $1/32$ 로 축소
 - Atrous 합성곱 적용: 출력이 입력의 $1/8$ 로 축소
 - $\Rightarrow$ 특성 맵 크기가 기존 대비 4배 보존



수용 영역 (Receptive Field)

: 외부 자극이 전체가 아닌 **특정 영역에만 영향**을 준다는 의미

- 영상에서 특정 픽셀은 주변 픽셀과 연관성이 높고, 거리가 멀어질수록 영향 ↓
- 전체 영역에 동일한 중요도를 주는 대신 **특정 범위를 한정해 처리** → 효과적인 훈련
- 이미지 분할에서 높은 성능을 내려면 수용 영역의 크기가 중요 (특징을 찾는 범위가 넓어지므로)

한눈에 정리

모델	핵심 아이디어	해결한 문제
ResNet	레지듀얼 블록 + 숏컷 ($H(x)=F(x)+x$)	깊은 망의 성능 저하 / 기울기 소멸
병목 블록	$1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$	깊이 증가 시 파라미터 폭증
R-CNN	선택적 탐색 + CNN	슬라이딩 윈도우의 비효율
SPP	크기 무관 입력 + 고정 크기 풀링	입력 크기 고정(crop/warp) 문제
Fast R-CNN	RoI 풀링	박스마다 CNN을 돌리는 속도 문제
Faster R-CNN	RPN + 앵커	느린 외부 선택적 탐색
FCN	완전연결층 → 1×1 합성곱	위치 정보 소실 / 입력 크기 제약
합성곱 & 역합성곱	업 샘플링(역합성곱)	FCN의 해상도 저하
U-Net	수축/확장 경로 + 병합	속도, 컨텍스트-지역화 트레이드오프
PSPNet	피라미드 풀링 모듈	다양한 영역의 컨텍스트 정보 활용
DeepLab	Atrous 합성곱 + ASPP	해상도 손실, 수용 영역 확장